# Project — Learning Worlds

Make it learn.


## 1 Objective

Working **individually or in pairs**, design a small interactive or dynamic system in which deep learning is used to learn something meaningful from data. You are free to choose the environment, task, data representation, objective, model, and training method.

The learned component may, for example, control an autonomous agent, predict the evolution of a system, imitate a behavior, classify or detect elements of the environment, generate content, or learn a useful representation.

The goal is not necessarily to achieve perfect performance, but to demonstrate a coherent process: problem design, learning, experimentation, evaluation, and critical analysis. Possible topics include cellular automata, survival or exploration games, Snake, a simplified Pac-Man or Tower Defense, artificial ecosystems, procedural generation, prediction or reconstruction tasks, and multi-agent simulations. Other original ideas are welcome, subject to approval.

## 2 Minimum Requirements

The project must include:

- a dynamic or interactive system with a visual representation;
- a clearly defined learning problem;
- clearly defined inputs, outputs, and objectives;
- a trained deep learning model;
- a quantitative evaluation protocol;
- a comparison with at least one relevant non-deep-learning baseline.

The baseline may be random, rule-based, heuristic, manually controlled, based on a classical algorithm, or any other relevant reference method.

The deep learning component may use supervised learning, imitation learning, reinforcement learning, self-play, representation learning, generative learning, evolutionary learning, or a justified combination of methods.

If you need inspiration or a starting point, you may:

- develop your own environment from scratch;
- build upon an existing game engine or open-source project;
- use an existing learning or simulation environment such as Gymnasium, MiniGrid, $\mu$RTS, or PettingZoo;
- adapt a simple existing game or simulation (e.g., Snake, Pac-Man, Cellular Automata, or similar).

These options are only suggestions. You are free to choose any other appropriate starting point, provided that your own contributions are clearly identified.

## 3 Expected Work

Your work should cover the following stages:

1. define the problem, system, inputs, outputs, and learning objective;
2. implement a functional system and a relevant baseline;
3. design and train a deep learning model;
4. define evaluation metrics and conduct several experiments;
5. compare the learned approach with the baseline;
6. analyze the results, observed behaviors, limitations, and possible improvements.

Formulate at least one hypothesis and evaluate it experimentally by modifying a single aspect of your approach while keeping the others unchanged. It can concern the DL architecture, input representation, objective or loss function, training duration, memory, data or environment diversity, model depth, or another relevant design choice. You may also study robustness and generalization to unseen situations.

Present the results with appropriate evidence: plots, tables, statistics, screenshots, or videos.

# 4 Deliverables

Submit a link to a Git repository containing:

- the complete source code;
- a `README` with installation, execution, and usage instructions;
- the commands required to run a demonstration and reproduce the evaluation;
- the trained model weights, when their size permits, or instructions to download them;
- a concise report;
- a short video or replay demonstrating the resulting system.

The repository should be self-contained and executable on another machine without significant modifications.

# 5 Report and Presentation

The report (maximum 2 pages) should briefly describe:

- the problem and system;
- inputs, outputs, objective, and baseline;
- the model and learning method;
- the experimental protocol and results;
- the interpretation of results, limitations, and future improvements.

Do not merely describe the code: justify your design choices and interpret the results.

The presentation must include the project concept, a demonstration, the model and training method, a comparison with the baseline, and a discussion of the results and limitations. The demonstration may be live or prerecorded.

# 6 Assessment

| **Criterion** | **Points** |
|---|---|
| System design, originality, and overall integration | /4 |
| Deep learning approach (model, training methodology, and justification) | /6 |
| Experimental methodology and evaluation | /4 |
| Analysis, discussion, and critical interpretation of the results | /3 |
| Implementation quality, reproducibility, and presentation | /3 |
| **Total** | **/20** |

Raw performance alone is not sufficient. An imperfect model can receive an excellent grade if the methodology is rigorous, the experiments are relevant, and the limitations are analyzed honestly. Creativity, scientific rigor, and the quality of the analysis are considered more important than achieving the highest score.

Happy coding!